

Rapport Technique

Systeme de Gestion des Auditoires et des Horaires

Par KINTAUDI NDINDA BENJAMINE ET INAKA LA JOIE

7 mai 2026

Table des matières

1	Introduction	3
2	Objectifs fonctionnels	3
2.1	Objectif general	3
2.2	Contraintes respectees	3
3	Architecture du projet	3
3.1	Arborescence logique actuelle	3
3.2	Separation des responsabilites	4
4	Modele de donnees JSON	4
4.1	Fichiers utilises	4
4.2	Relations fonctionnelles	4
4.3	Referentiels pedagogiques	4
5	Logique metier principale	5
5.1	Regles de reservation	5
5.2	Gestion des conflits	5
5.3	Attribution intelligente des salles	5
5.4	Generation automatique du planning	5
6	Interfaces et parcours utilisateur	6
6.1	Espace public	6
6.2	Espace administrateur	6
7	Securite et controle d'accès	6
7.1	Authentification	6
7.2	RBAC (Role-Based Access Control)	6
7.3	Regle de securite specifique	7
8	Qualite et robustesse	7
8.1	Validation des donnees	7
8.2	Integrite des ecritures	7
8.3	Verification technique	7
8.4	Ameliorations d'interface	7
9	Limites actuelles	8
10	Difficultes rencontrees	8
10.1	Gestion des droits et permissions	8

10.2 Architecture JSON et coherence des donnees	8
10.3 Theme visuel et coherence CSS	8
10.4 Filtrage et recherche cote client	8
10.5 Donnees pedagogiques et specialisations	9
10.6 Organisation du code et maintenance	9
11 Conclusion	9

1 Introduction

Ce document presente de maniere detaillee le travail realise pour concevoir et developper une application complete de gestion des auditorios. L'application permet de planifier les cours, reserver les salles, eviter les conflits d'occupation et optimiser l'affectation des ressources selon la capacite des auditorios. Elle integre aussi un espace administrateur securise, des droits par module, un theme sombre harmonise et des fonctions de recherche et filtrage dans les listes principales.

Le projet est construit en **PHP procedural** (sans framework), avec des vues HTML/CSS integrees dans les fichiers PHP et un stockage des donnees en **JSON**.

2 Objectifs fonctionnels

2.1 Objectif general

Mettre en place un systeme academique permettant de :

- planifier les cours ;
- reserver les salles ;
- attribuer les salles selon la capacite des groupes ;
- bloquer les conflits (meme salle, meme creneau) ;
- rendre le planning coherent, lisible et exploitable.

2.2 Contraintes respectees

- aucun framework PHP ;
- persistance uniquement via des fichiers JSON ;
- logique metier centralisee dans des fonctions reutilisables ;
- interfaces admin et publiques distinctes ;
- controle d'accès administrateur avec gestion de droits.

3 Architecture du projet

3.1 Arborescence logique actuelle

- `index.php` : page publique principale (accueil + aperçus) ;
- `config/config.php` : configuration globale de l'application ;
- `data/*.json` : base de donnees JSON ;
- `includes/` : fonctions, header, footer ;

- **admin/** : espace administrateur (CRUD + planning + securite + gestion admins);
- **actions/** : endpoints de traitement (ajout, generation, reset, etc.);
- **auth/login.php** et **auth/logout.php** : authentication;
- **pages/stats.php** : statistiques et disponibilites;
- **assets/css/style.php**, **assets/js/app.js** : ressources front.

3.2 Separation des responsabilites

- **Config** : constantes application + resolution d'URL;
- **Fonctions metier** : lecture/criture JSON, validations, planning, permissions;
- **Pages** : presentation et formulaires;
- **Actions** : traitement des soumissions et redirections;
- **Interface front** : theme sombre, ergonomie unifiee, filtres de tableau reutilisables.

4 Modele de donnees JSON

4.1 Fichiers utilises

Fichier	Contenu
salles.json	id, designation, capacite
promotions.json	id_promotion, libelle, effectif_total
options.json	id_option, libelle, promotion_parente, effectif
cours.json	id_cours, intitule, volume_horaire, promotion_option
planning.json	creneau, salle, cours, groupe
admin.json	liste des administrateurs, hash de mot de passe, statut actif, permissions

4.2 Relations fonctionnelles

- un **cours** cible soit une promotion, soit une option (champ **promotion_option**);
- une **option** depend d'une promotion parente;
- une ligne de **planning** relie un creneau, une salle, un cours et un groupe.

4.3 Referentiels pedagogiques

Les donnees de promotion et d'option ont ete enrichies pour mieux représenter le parcours academique :

- promotions de base : L1, L2, L3, L4, Master et Doctorat ;
- options rattachees aux niveaux L3 et L4 pour les specialisations ;
- affichage explicite du code et du libelle pour faciliter la selection.

5 Logique metier principale

5.1 Regles de reservation

Lors de l'ajout (manuel ou automatique) d'une seance :

- tous les champs doivent etre remplis ;
- le groupe doit exister ;
- la salle doit etre libre sur le creneau ;
- le groupe ne doit pas deja etre occupe sur le meme creneau ;
- la capacite de la salle doit etre suffisante.

5.2 Gestion des conflits

Deux types de conflits sont verifies :

- **conflit salle** : meme salle + meme creneau ;
- **conflit groupe** : meme groupe + meme creneau.

5.3 Attribution intelligente des salles

L'algorithme trie les salles par capacite croissante et prend la premiere salle disponible capable d'accueillir le groupe. Cela minimise le gaspillage des grandes salles et reserve les capacites elevees aux grands effectifs.

5.4 Generation automatique du planning

La generation suit une logique en plusieurs etapes :

1. tri des cours par taille de groupe (du plus grand au plus petit), puis volume horaire ;
2. decoupage du volume horaire en sessions (par bloc d'environ 2h) ;
3. recherche d'un creneau sans conflit ;
4. recherche de la meilleure salle disponible ;
5. ecriture des seances dans `planning.json` ;
6. retour des erreurs si certaines sessions sont impossibles a placer.

6 Interfaces et parcours utilisateur

6.1 Espace public

- page d'accueil : KPIs, planning du jour, planning global, acces rapide au planning ;
- page statistiques : disponibilite des salles, occupation par jour, capacites globales ;
- interface alligee sur un theme sombre avec espaces visuels harmonises.

6.2 Espace administrateur

- dashboard (indicateurs de synthese) ;
- CRUD salles, cours, promotions, options ;
- planning : filtres, vue hebdomadaire, ajout, modification, suppression, generation auto, reset ;
- recherche et filtrage dans les tableaux administratifs ;
- changement de mot de passe ;
- gestion des administrateurs et des droits.

7 Securite et controle d'accès

7.1 Authentification

- connexion via `auth/login.php` ;
- deconnexion via `auth/logout.php` ;
- mots de passe stockes en hash (`password_hash/password_verify`) ;
- session PHP pour maintenir l'etat authentifie.

7.2 RBAC (Role-Based Access Control)

Chaque administrateur possede un ensemble de permissions booleennes :

- `manage_rooms`
- `manage_courses`
- `manage_promotions`
- `manage_options`
- `manage_planning`
- `manage_admins`

Ces permissions sont appliquees a la fois sur :

- l’affichage des menus ;
- l’accès aux pages ;
- l’exécution des actions serveur.

La gestion des administrateurs reste reservee au super-administrateur, tandis que les autres admins peuvent gerer completement leurs modules autorises (ajout, modification, suppression).

7.3 Regle de securite specifique

Un administrateur connecte ne peut pas se retirer lui-meme le droit `manage_admins`. Cette protection evite de perdre la capacite de gouvernance des comptes administrateurs.

8 Qualite et robustesse

8.1 Validation des donnees

La validation est appliquee sur les champs critiques (vide, types, coherence metier). Les erreurs sont restituees a l’utilisateur via des messages flash clairs.

8.2 Integrite des ecritures

Les suppressions/modifications de planning sont basees sur des cles logiques (creneau+salle+cours+groupe), et non sur des index visuels, ce qui reduit les risques d’erreur avec les filtres.

8.3 Verification technique

Les fichiers PHP ont ete verifiees regulierement avec `php -l` pour detecter rapidement les erreurs de syntaxe durant l’evolution du code. Au moment de la redaction de ce rapport, l’application ne presente aucune erreur PHP detectee par la verification automatique.

8.4 Ameliorations d’interface

Les derniers ajustements visuels incluent :

- passage a un theme sombre avec palette verte foncee ;
- espacement renforce entre les blocs de statistiques et les sections de contenu ;
- suppression du soulignement des liens de navigation ;
- bouton d’accès administrateur masque pour les comptes non super-administrateurs ;
- mise en avant plus lisible des promotions parentes et des specialisations.

9 Limites actuelles

- stockage JSON adapte aux petites/moyennes charges mais moins adapte au multi-utilisateur intensif;
- absence de transactions fortes comme dans une base relationnelle;
- historique/audit des actions administrateur non encore implemente;
- recherche et filtrage encore simples cote client, sans indexation avancee.

10 Difficultes rencontrees

10.1 Gestion des droits et permissions

La principale difficulte a ete la transition d'une logique basee sur les roles (super-admin) vers un systeme de permissions granulaires par module. L'approche initiale avait mis en place des restrictions strictes qui empechaient les administrateurs reguliers d'accéder aux pages CRUD meme s'ils possedaient les permissions appropriees. La solution a necessite de restructurer completement la logique de verification d'accès et de la rendre modulaire pour chaque permission (`manage_rooms`, `manage_courses`, etc.).

10.2 Architecture JSON et coherence des donnees

L'absence de transactions et de contraintes fortes du JSON a complique la gestion de la coherence entre fichiers (`promotions.json`, `options.json`, `planning.json`). Lors de la suppression d'une promotion, il fallait nettoyer les cours associes et leurs occurrences dans le planning. Cette gestion manuelle des relations a augmente le risque de donnees orphelines et a necessite des fonctions de validation poussees.

10.3 Theme visuel et coherence CSS

La transition vers un theme sombre avec palette verte a necessite de revoir systematiquement tous les fichiers PHP pour harmoniser les couleurs, les espacements et les bordures. Le challenge principal etait de maintenir la lisibilite en mode sombre tout en conservant une interface professionnelle, en particulier pour les tableaux administratifs et les formulaires.

10.4 Filtrage et recherche cote client

L'implementation de la recherche et du filtrage dans les listes administratives s'est averee complexe en raison de la nature procedurale du JavaScript et de la necessite de maintenir la synchronisation avec les donnees JSON. Les filtres devaient fonctionner sans

rechargement de page tout en restant performants sur des tableaux contenant des centaines de lignes.

10.5 Données pedagogiques et specialisations

L'enrichissement des donnees de promotion et d'option (ajout de Master, Doctorat et specialisations L3/L4) a necessite de repenser la structure de `promotions.json` et `options.json`. La retroactivite sur les donnees existantes a demande une strategie d'evolution sans casser les courses deja associees.

10.6 Organisation du code et maintenance

La croissance progressive des fichiers action a necessité une reorganisation des scripts de maintenance dans un dossier dedie. Distinguer entre scripts de maintenance (usage exceptionnel) et fichiers application critiques a aide a clarifier le perimetre et les responsabilites.

11 Conclusion

Le projet repond aux exigences centrales : planification, reservation, prevention des conflits, respect des capacites, optimisation de l'affectation et administration securisee. La conception en PHP procedural et JSON est respectee, avec une architecture claire, maintenable et evolutive.

Ce travail constitue une base operationnelle solide pour la gestion academique des auditories et des horaires, tout en laissant de la place a des extensions futures. Les corrections recentes ont permis de stabiliser les droits par module, d'ameliorer la lisibilite de l'interface et de rapprocher le rapport de l'etat reel de l'application.